Cheatsheet AZ-305 — 08 · Compute and containers

1. The PaaS ↔ IaaS axis: when a VM is required

Requirement in the wording	Answer
Administrative access to the OS, install system-level dependencies	**VM / VMSS** (no App Service, not even Isolated/ASE, gives OS access)
COM component registered on the host	**VM / VMSS**
Legacy software with OS dependencies	**VM / VMSS**
Full .NET Framework + writing temp files + Event Log, minimum maintenance	**App Service (Windows)** is enough
Standard web app/API, minimum maintenance and cost	**App Service**
Not sharing physical hardware with other customers	**Azure Dedicated Hosts** (VMs)
Survive a data center	**Availability Zones** (VMSS across 2+ zones)
Survive a region	Two regions + Traffic Manager / Front Door

2. App Service: plans and ASE

Tier	Dedicated instances	Autoscale / slots	VNet Integration (outbound)	Private Endpoint (inbound)	Isolation
Free / Shared	✗	✗	✗	✗	—
Basic	✓	✗	✓ (v2)	✓	Multi-tenant
Standard	✓	✓	✓	✓	Multi-tenant
Premium (v2/v3)	✓ more CPU/RAM	✓ more slots	✓	✓	Multi-tenant
Isolated (v2)	✓	✓ up to 200 inst.	✓	✓	**Only inside an ASE**

- **App Service Environment (ASE):** full App Service deployment **inside your VNet** (private IP, no public IP on an ILB ASE, all traffic under your NSGs/UDRs/firewall, up to 200 instances). **Still PaaS: no OS access.** Very expensive → only for total isolation, compliance or scale.
- **Private traffic App Service → SQL:** VNet Integration (**delegated**, exclusive subnet) + SQL Private Endpoint (another subnet) + Private DNS zone → **minimum 2 subnets**.
- **Regional redundancy:** a plan lives in one region; multi-region = two deployments + Front Door / Traffic Manager.
- **Deployment slots** (Standard+): swap with no downtime. **Health check** to remove unhealthy instances.
- Repeatable configuration with history → **Bicep/ARM in Azure Repos** (not ZIP, not images).

3. Azure Functions: plans

Plan	Scaling	VNet Integration	Cold start	Cost	When
Consumption	Event-driven, to zero	✗ (limited)	Yes	Minimum	One-off code without private networking
Flex Consumption	Event-driven, to zero	✓	Reduced	Low	Serverless with VNet
Premium	Event-driven, pre-warmed instances	✓	No	Medium	**Access a private IP** + reasonable cost
Dedicated (App Service plan)	Manual/autoscale of the plan	✓	No	Fixed	Reuse an existing plan

- **Durable Functions:** Client (starts) · **Orchestrator** (defines the flow, **durable timers**, conditionals) · Activity (work) · Entity (state).
- **Authorization level:** Anonymous (public) · Function (key) · Admin (master key). Public read-only → **Anonymous + GET only**.
- Queue Storage trigger migrated to AKS → **KEDA + HPA**.

4. Containers: ACI vs Container Apps vs AKS

	Container Instances	Container Apps	AKS
Unit	One container / container group	App with replicas	Full Kubernetes cluster
Automatic scaling	✗	✓ 0→N (HTTP, CPU, **events/KEDA**)	✓ HPA, KEDA, cluster autoscaler
Scale to zero	Only by deleting	✓	With KEDA
Managed ingress / TLS	✗	✓	Ingress controller you deploy
Revisions / canary	✗	✓	With tools (Flux, Istio)
Microservices / Dapr	✗	✓ built in	Add-on
Cluster control	—	Hidden	**Full** (kubectl, Helm, CRDs, networking)
Startup	Seconds	Seconds	Minutes (nodes)
Billing	Per second of CPU/RAM	Per consumption	Nodes (VMs)
Use case	Jobs, one-off tasks, **AKS burst (virtual nodes)**	APIs, web apps, microservices without managing K8s	Kubernetes workloads, fine-grained control

- **Service Fabric:** Microsoft's microservices orchestrator, **hybrid on-prem + Azure**, independent updates, self-healing, hyperscale.
- **Azure Batch:** parallel compute with its own scheduler. **CycleCloud:** HPC clusters with a **third-party scheduler** (Slurm, PBS, LSF).

- **Container Registry Premium: geo-replication** of images for multi-region AKS; CMK; private link.

5. AKS: pieces that appear in the exam

Piece	What it does	Signal
Ingress controller (NGINX, AGIC)	Single L7 entry point: paths, hosts, TLS, **mTLS**, non-standard ports; avoids one LB per service	"expose multiple APIs", "non-default ports", "mTLS", "min cost"
Service LoadBalancer	One Azure LB per service, L4	Only if a single service
ExternalName service	DNS alias to something external	Distractor
HPA	Scales pods by CPU/RAM or external metrics	With KEDA
KEDA	Scales pods by **events** (queues, Service Bus, Event Hubs), 0↔1; creates the HPA	"same scaling as Functions Consumption"
Cluster autoscaler	Scales **nodes**; works with kubenet and Azure CNI	"nodes", "capacity"
Virtual nodes / Virtual Kubelet	Pods on ACI for burst; **Azure CNI only**	Ruled out if they say "kubenet"
Dapr	Building blocks: **state management**, pub/sub, service invocation, secrets	"state management"
Flux	GitOps: syncs the cluster with a repo	"deploy from Git"
Istio	Service mesh: mTLS between pods, canary, traffic	"traffic between services"
Azure Policy add-on	Gatekeeper/OPA (EnforceOPAConstraint / Rego)	"pod governance"
kubenet vs Azure CNI	kubenet: pod IPs NATed; CNI: VNet IPs (more IPs, direct integration)	"pods reachable from VNet" → CNI

- Front Door → 2 AKS clusters in 2 regions with centralized SSL.
- APIM + AKS same VNet, private IP, mTLS, rate limit (APIM options) → Premium Internal.

6. VMs and VMSS: exam rules

Topic	Rule
SLA	Single VM with Premium SSD 99.9 % · Availability Set 99.95 % · Availability Zones 99.99 %
Availability Set	Fault / update domains, same datacenter; does not protect against datacenter failure
Availability Zones	Separate datacenters within the region; "data center failure"
VMSS	**Horizontal** scaling; autoscale with metrics, duration, cooldown, **profiles** by schedule/date; software at provisioning with **Custom Script / DSC extension** in extensionProfile + uploaded script
Series	D/Ds general purpose (default and cost), E memory, F compute, L local disk, M massive memory, N GPU (NC compute, NV visual), H HPC; **"s" = Premium storage**; SR-IOV = Accelerated Networking
Disks	OS: Premium SSD **v1** (v2 not valid for OS); SQL: log **None**, data **ReadOnly**
Dedicated Hosts	Non-shared physical hardware
Confidential VMs	Data in use encrypted (TEE)
Scale with no downtime	Horizontal + VMSS + Availability Set/Zones
JIT VM access	Defender for Cloud opens management ports temporarily (it is not PIM)
VMware migration	Azure Migrate appliance (discovery) / AVS only if they want to stay on VMware

7. Decision tree

```
Which hosting?
├─ Needs OS / COM / host dependencies ..... VM / VMSS (zones for datacenter)
├─ Standard web/API, min maintenance ..... App Service (Standard/Premium)
├─ Same but no public IP / total isolation ..... ASE (still no OS)
├─ Event-driven code ..... Functions (Premium if private IP)
├─ One-off container / job ..... Container Instances
├─ Microservices without managing K8s ..... Container Apps
├─ Kubernetes with control ..... AKS
├─ Hybrid on-prem + Azure microservices ..... Service Fabric
├─ HPC with third-party scheduler ..... CycleCloud
└─ Managed parallel compute ..... Batch
```

8. Signals → answer

Wording	Answer
"administrators access to the OS"	VMs / VMSS (never App Service)
"custom COM component" + "data center unavailable" + "min cost"	VMSS + LB across 2 availability zones
"Isolated plan" + "redundancy if region fails"	Does not meet it: one region, no OS
".NET web service, temp files, event log, min cost"	App Service web app
"Event Grid + C# + private IP + min cost"	Functions Premium
"state management" (AKS)	Dapr

Wording	Answer
"same scaling as Consumption" (AKS, kubenet)	KEDA + HPA
"expose APIs on non-default ports + mTLS + min cost"	Ingress controller
"image replicated to AKS in multiple regions"	Container Registry Premium (geo-replication)
"HPC + third-party scheduler"	CycleCloud
"on-prem + Azure microservices, self-healing"	Service Fabric
"define config of 50 App Services + version history"	Bicep in Azure Repos
"15,000 IOPS + SR-IOV + min cost"	DS series

9. Mistakes to avoid

1. ASE/Isolated as the answer to "OS access".
2. Two regions when they ask to survive a data center and minimize cost → zones.
3. Virtual nodes with kubenet.
4. HPA alone (CPU) to replicate Functions Consumption → KEDA + HPA.
5. Container Instances for an API with autoscaling → Container Apps.
6. Vertical scaling on VMSS.